Inception-of-Things (IoT) — Day-by-Day Plan

A 10-day execution plan for the **mandatory part** of the subject (Parts 1–3), split into what to **learn** and what to **build** each day, plus an optional bonus day. Based on your current repo state, Parts 1 and 2 already have work done — those days are framed as *review/harden* rather than *build from scratch*, so you don't lose time re-doing what already works.

Each day assumes roughly a focused work session (2–4 hours). Slower or faster is fine — the order matters more than the exact pacing.

Day 1 — Environment & fundamentals

Learn - Linux shell scripting basics: variables, `if`, functions, `set -e`, running scripts as root vs user. - Networking basics: what a network interface is (`eth0`, `enp0s8`...), private/host-only networks, static IP addressing, `ip a` / `ifconfig`. - SSH key-based (passwordless) authentication: `ssh-keygen`, `authorized_keys`.

Build - Install/verify host tooling: VirtualBox (or your chosen provider), Vagrant, `kubectl`, `git`. - Confirm your Git repository is initialized with the expected folder layout at the root: `p1/`, `p2/`, `p3/` (each will need a `scripts/` and `confs/` subfolder).

Steps 1. `vagrant --version`, `VBoxManage --version`, `kubectl version --client`, `git --version`. 2. Generate an SSH key pair if you don't already have one: `ssh-keygen -t ed25519`. 3. Create the base folders: `mkdir -p p1/scripts p1/confs p2/scripts p2/confs p3/scripts p3/confs`.

Day 2 — Vagrant + Kubernetes core concepts

Learn - `Vagrantfile` syntax: `config.vm.define`, `config.vm.network "private_network", ip: ...`, `config.vm.provider`, `config.vm.provision "shell"`, inline vs script-file provisioning. - Kubernetes core concepts: **cluster**, **node**, **pod**, and what "control plane" means.

Build - `p1/Vagrantfile` defining **two** VMs: `<login>S` (server) and `<login>SW` (server-worker).

Steps 1. Pick a distro + find its latest Vagrant box on Vagrant Cloud. 2. Write two `config.vm.define` blocks with correct `hostname`, static `private_network` IP (`192.168.56.110` for S, `192.168.56.111` for SW), and 1 CPU / 512–1024 MB RAM. 3. `vagrant up` and confirm both VMs boot and get their static IPs (`ip a` inside each). 4. Set up passwordless SSH into both (`vagrant ssh-config` + your own key, or a provisioning script that injects your public key).

Day 3 — K3s server & agent (Part 1 core)

Learn - K3s architecture: single binary, embedded containerd, SQLite by default, server mode vs agent mode, what the **node token** is used for. - `kubectl` basics: `get`, `describe`, `apply -f`, `logs`, `-n` for namespaces.

Build - `p1/scripts/server.sh` (K3s server/controller install) and `p1/scripts/worker.sh` (K3s agent install joining the server via token).

Steps 1. In `server.sh`: `curl -sfL https://get.k3s.io | sh -`, then read the token from `/var/lib/rancher/k3s/server/node-token`. 2. Get that token onto the worker (shared file, provisioning variable, or manual copy into `p1/k3s-token` as you're already doing). 3. In `worker.sh`: install K3s in agent mode with `K3S_URL=https://192.168.56.110:6443` and `K3S_TOKEN=<token>`. 4. Wire both scripts into the `Vagrantfile` as shell provisioners.

Day 4 — Finish & validate Part 1

Learn - Reading `kubectl get nodes -o wide` output: roles, versions, internal IPs, ready status. - Common K3s troubleshooting: checking `systemctl status k3s` / `k3s-agent`, journal logs.

Build - Nothing new — **harden and verify** the existing `p1/` setup end-to-end.

Steps 1. `vagrant destroy -f && vagrant up` from a clean state — the whole thing must be reproducible. 2. From S: `k get nodes -o wide` — S must show `control-plane,master` role, SW must show as an agent node, both `Ready`, correct internal IPs (`.110` / `.111`). 3. Confirm passwordless SSH into both machines from the host. 4. Commit `p1/Vagrantfile`, `p1/scripts/`, `p1/confs/` (if any) — this part is done.

Day 5 — Services & Ingress deep dive (Part 2 setup)

Learn - Kubernetes **Deployment** vs **ReplicaSet** vs **Pod**. - **Service** (ClusterIP) — stable internal networking for pods. - **Ingress** + Ingress Controller (K3s ships **Traefik** by default) — routing by `Host` header.

Build - `p2/Vagrantfile` for a **single** VM (`<login>S`, K3s server mode only — no agent this time).

Steps 1. Write `p2/Vagrantfile` (same static IP `192.168.56.110`, same hostname pattern `<login>S`). 2. Write/reuse `p2/scripts/install_k3s.sh` for server-mode K3s install. 3. `vagrant up` and confirm the single-node cluster is `Ready` with Traefik running (`k get pods -n kube-system`).

Day 6 — Build the three apps (Part 2 core)

Learn - Ingress default-backend / catch-all routing (no `host` restriction). - How to test Ingress rules without real DNS, using `curl -H "Host: ..."`.

Build - `p2/confs/app1/`, `app2/`, `app3/`: each with `deployment.yml`, `service.yml`, `ingress.yml`.

Steps 1. app1 → `deployment.yml` (1 replica) + `service.yml` (ClusterIP) + `ingress.yml` (`Host: app1.com`). 2. app2 → same but `replicas: 3`, `Host: app2.com`. 3. app3 → same but its `ingress.yml` is the **default/catch-all** (any other Host value lands here). 4. `kubectl apply -f` everything (or bake it into `p2/scripts/setup.sh`). 5. Test: `curl -H "Host: app1.com" http://192.168.56.110 curl -H "Host: app2.com" http://192.168.56.110 curl http://192.168.56.110 # falls through to app3` 6. Confirm app2 has 3 running pods (`k get pods`) and commit `p2/`.

Day 7 — Docker & K3d fundamentals (Part 3 setup)

Learn - Docker basics: images, containers, tags, Docker Hub as a registry. - Why **K3d** needs Docker: it runs a full K8s cluster *as Docker containers*, unlike K3s which installs directly on a VM/host as a systemd service. Understand the K3s-vs-K3d distinction clearly — you'll be asked about it during the defense.

Build - `p3/scripts/install.sh`: installs Docker, K3d, and `kubectl` (must be runnable live during the defense).

Steps 1. Write the install script (Docker's official convenience script or your distro's package manager, then `curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash`). 2. Run it on the VM (no more Vagrant provisioning needed for this part — same VM, direct install). 3. Create the cluster: `k3d cluster create <name>`. 4. Verify: `kubectl get nodes` should show K3d's node(s) as `Ready`.

Day 8 — GitOps & Argo CD concepts + install

Learn - **GitOps**: your Git repo is the single source of truth; a controller (Argo CD) continuously reconciles the live cluster to match Git. - Argo CD **Application** custom resource: repo URL + path + target revision + destination namespace, manual vs automatic sync.

Build - `argocd` and `dev` namespaces; Argo CD installed in `argocd`; a **public GitHub repo** containing your `p3` manifests (repo name must include a group member's login).

Steps 1. `kubectl create ns argocd` and `kubectl create ns dev`. 2. Install Argo CD: `kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml`. 3. Get access to the UI/API (port-forward is simplest to start: `kubectl port-forward svc/argocd-server -n argocd 8080:443`). 4. Create the public GitHub

repo, push a `deployment.yaml` (+ service/ingress if needed) referencing your chosen app image. 5. Decide the app source: Wil's pre-made `wil42/playground` (Docker Hub, port 8888) **or** your own app pushed to a public Docker Hub repo tagged `v1` / `v2` with a visible difference.

Day 9 — Finish Part 3: sync, verify, switch version

Learn - Image **tagging** (`v1`, `v2`) to represent two versions of the same app, and how Argo CD picks up a change in the tracked Git path.

Build - An Argo CD `Application` resource wired to your GitHub repo, deploying into `dev`, running **v1** first, then switched to **v2** via a Git push.

Steps 1. Create the Argo CD `Application` (repo URL, path, target revision, destination namespace `dev`). 2. Sync and verify v1: `curl http://localhost:8888/` (through your port-forward/ingress) reports the v1 message. 3. Edit `deployment.yaml` in your GitHub repo to the `v2` image tag, commit + push. 4. Trigger/observe the sync in Argo CD (manual sync, or wait for auto-sync). 5. Verify v2: `curl` again — response now reflects v2. Practice this full loop end-to-end since it's exactly what gets demonstrated live during the defense. 6. Commit `p3/scripts/`, `p3/confs/`.

Day 10 — Full review & defense rehearsal

Learn - Nothing new — this day is about being able to **explain** every choice you made (why K3s vs K3d, why this Ingress setup, what GitOps buys you) since the defense is oral.

Build - Nothing new — final reproducibility pass across all three parts.

Steps 1. From a totally clean state, `vagrant destroy -f` on `p1/` and `p2/`, and `k3d cluster delete` on `p3/`; rebuild everything from scripts only, timing how long each part takes. 2. Re-run every verification command from Days 4, 6, and 9 and confirm identical results. 3. Rehearse showing the Ingress live in Part 2 (the subject deliberately doesn't screenshot it — you must demo it). 4. Rehearse the v1 → v2 GitOps update live in Part 3. 5. Double-check folder structure matches the expected layout exactly (`p1/`, `p2/`, `p3/`, each with `scripts/` and `confs/`).

Day 11 (optional) — Bonus: GitLab

> Only worth attempting if the mandatory part is **100% flawless** — the bonus isn't graded otherwise.

Learn - Helm basics (installing charts, values files) — likely the simplest path to a local GitLab instance.

Build - A local GitLab instance in a dedicated `gitlab` namespace, wired into the same Part 3 flow (Argo CD tracking a repo hosted on your local GitLab instead of GitHub).

Steps 1. `kubectl create ns gitlab`. 2. Install GitLab (Helm chart from the official GitLab chart repo) into that namespace. 3. Re-point (or duplicate) your Argo CD `Application` to a repo hosted on your local GitLab. 4. Re-run the full v1 → v2 sync test against this new source. 5. Put everything needed in a `bonus/` folder (`scripts/` , `confs/`) at the repo root.

Quick reference — folder structure expected at the end

```
p1/           Vagrantfile, scripts/, confs/  (K3s server + agent via Vagrant)
p2/           Vagrantfile, scripts/, confs/  (single K3s VM + 3 apps + Ingress)
p3/           scripts/, confs/               (K3d + Argo CD, no Vagrant)
bonus/        scripts/, confs/               (optional: GitLab)
```